Data Structures: Assignment-12

Name: Suraj Shah

Admission Number: I24AI012

1. Write a C or C++ program to sort the input array [12, 45, 33, 87, 56, 9, 11, 7,

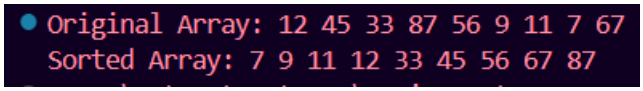
67] using the Bucket Sort algorithm with 7 buckets.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

void bucketSort(vector<int>& arr, int bucketCount) {
    if (arr.empty()) {
        return;
    }

    int min_value=*min_element(arr.begin(), arr.end());
    int max_value=*max_element(arr.begin(), arr.end());

    vector<vector<int>> buckets(bucketCount);

    for (int num: arr) {
        int index=(num-min_value)*bucketCount/(max_value-min_value+1);
        buckets[index].push_back(num);
    }

    arr.clear();
    for (auto bucket: buckets) {
        sort(bucket.begin(), bucket.end());
        arr.insert(arr.end(), bucket.begin(), bucket.end());
    }
}

int main() {
    vector<int> arr={12, 45, 33, 87, 56, 9, 11, 7, 67};
    int bucketCount=7;

    cout << "Original Array: ";
    for (int num: arr) {
        cout << num << " ";
```

```
    }
    cout << endl;

    bucketSort(arr, bucketCount);

    cout << "Sorted Array: ";
    for (int num: arr) {
        cout << num << " ";
    }
    cout << endl;
}
```

Output:



2. Write a C/C++ program that ask the user to enter 10 integer values. Use these values to construct a binary tree with 10 nodes. After constructing the tree, perform and display the inorder, preorder, and postorder traversals.

Code:

```
#include <iostream>
using namespace std;

struct Node{
    int data;
    Node *left;
    Node *right;

    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

Node *insert(Node *root, int val) {
    if (root==nullptr) {
        return new Node(val);
    }
    if (val<root->data) {
        root->left=insert(root->left, val);
    }
    else {
        root->right=insert(root->right, val);
    }
    return root;
}

void inorder(Node *root) {
    if (root!=nullptr) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

void preorder(Node *root) {
    if (root!=nullptr) {
        cout << root->data << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(Node *root) {
```

```
    if (root!=nullptr) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}

int main() {
    Node *root=nullptr;
    int value;

    cout << "Enter 10 integer values:\n";
    for (int i=0; i<10; i++) {
        cout << "Value: " << i+1 << ": ";
        cin >> value;
        root=insert(root, value);
    }

    cout << "\nInorder traversal: ";
    inorder(root);

    cout << "\nPreorder traversal: ";
    preorder(root);

    cout << "\nPostorder traversal: ";
    postorder(root);
}
```

Output:

```
Enter 10 integer values:
Value: 1: 12
Value: 2: 45
Value: 3: 33
Value: 4: 87
Value: 5: 56
Value: 6: 9
Value: 7: 11
Value: 8: 7
Value: 9: 67
Value: 10: 31

Inorder traversal: 7 9 11 12 31 33 45 56 67 87
Preorder traversal: 12 9 7 11 45 33 31 87 56 67
Postorder traversal: 7 11 9 31 33 67 56 87 45 12
```

3. Given the inorder and preorder traversals of a binary tree, write a C/C++

program to construct the binary tree.

Inorder: 1, 8, 19, 13, 25, 9, 5, 10, 4, 3

Preorder: 25, 8, 1, 13, 19, 5, 9, 4, 10, 3

Code:

```
#include <iostream>
using namespace std;

struct Node{
    int data;
    Node *left;
    Node *right;

    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

Node *buildTree(int pre[], int lpre, int hpre, int in[], int lin, int hin) {
    if (lpre<=hpre) {
        Node *root=new Node(pre[lpre]);
        int i=0;
        for (i=lin; i<=hin; i++) {
            if (pre[lpre]==in[i]) {
                break;
            }
        }
        int size=i-lin;
        root->left=buildTree(pre, lpre+1, lpre+size, in, lin, lin+size-1);
        root->right=buildTree(pre, lpre+size+1, hpre, in, lin+size+1, hin);
        return root;
    }
    return nullptr;
}

void inorderTraversal(Node* root) {
    if (root) {
        inorderTraversal(root->left);
        cout << root->data << " ";
        inorderTraversal(root->right);
    }
}

void preorderTraversal(Node* root) {
    if (root) {
        cout << root->data << " ";
        preorderTraversal(root->left);
```

```
        preorderTraversal(root->right);
    }
}

int main() {
    int inorder[]={1, 8, 19, 13, 25, 9, 5, 10, 4, 3};
    int preorder[]={25, 8, 1, 13, 19, 5, 9, 4, 10, 3};
    Node *root=nullptr;
    int n=sizeof(inorder)/sizeof(inorder[0]);

    root=buildTree(preorder, 0, n-1, inorder, 0, n-1);

    cout << "Inorder (from built tree): ";
    inorderTraversal(root);

    cout << "\nPreorder (from built tree): ";
    preorderTraversal(root);

    cout << endl;
}
```

Output:

```
Inorder (from built tree): 1 8 19 13 25 9 5 10 4 3
Preorder (from built tree): 25 8 1 13 19 5 9 4 10 3
```